Threads in Java and OCaml 5: the two sheets compared

Programming Paradigms, session 7. The ten rules of the two sheets set side by side: what never to write and what to write in each language, who checks, the runs, where the languages part, and the code.

What never to write, and what to write

Rule	Java: never	Java: write	OCaml 5: never	OCaml 5: write
1 every access, or none	protect some of the accesses to a piece of data.	one lock per piece of data, named beside it, and take it in every method that touches the data.	protect some of the accesses to a piece of data.	one lock per piece of data, named beside it, and take it in every method that touches the data.
2 two locked steps are not one	release a lock in the middle of an operation whose invariant spans the data it protects.	the invariant first; then choose the critical section so that the invariant is true at its start and at its end, and hold every lock the invariant needs for the whole of it.	release a lock in the middle of an operation whose invariant spans the data it protects.	the invariant first; then choose the critical section so that the invariant is true at its start and at its end, and hold every lock the invariant needs for the whole of it.
3 the plain cell is not atomic	write `n++` on a shared field, volatile or not.	an atomic cell for a counter or a flag; a lock when more than one word must change together.	`incr` a shared `ref`.	an atomic cell for a counter or a flag; a lock when more than one word must change together.
4 wait in a loop	`if (cond) wait();`	`while (!ready) wait();`	`if cond then Condition.wait …`	`while not ready do Condition.wait c m done`
5 wake everyone	wake one thread when more than one kind of thread waits on the same condition.	`notifyAll` by default; one condition per kind of waiter and `notify` only with the argument written down.	wake one thread when more than one kind of thread waits on the same condition.	`Condition.broadcast` by default; one condition per kind of waiter and `signal` only with the argument written down.
6 one lock order	take lock B while holding A in one	the order down, by an identifier or an address,	take lock B while holding A in one place and A while	the order down, by an identifier or an address, and a

	place and A while holding B in another.	and a helper that takes two locks in that order.	holding B in another.	helper that takes two locks in that order.
7 compare what you read	pass a literal or a freshly built value as the expected argument of a compare-and-set.	`Integer v = r.get();` `r.compareAndSet(v, f(v))`, and loop on failure. `AtomicInteger` compares by value, because the number is the object.	pass a literal or a freshly built value as the expected argument of a compare-and-set.	`let v = Atomic.get r in …` `Atomic.compare_and_set r v (f v)`, and loop on failure. `int Atomic.t` compares by value, because the number is the object.
8 a run proves nothing	conclude from a run, or from a hundred, that a shared structure is right.	the invariant and the point where each operation takes effect; then test with many threads, many operations, many runs, to catch what the argument missed.	conclude from a run, or from a hundred, that a shared structure is right.	the invariant and the point where each operation takes effect; then test with many threads, many operations, many runs, to catch what the argument missed.
9 a write another thread must see	pass a flag or a result between threads through a plain field.	`volatile` or an atomic for a flag; a lock, a queue or a join for anything larger. A value published before a volatile write is visible after a volatile read of the same field.	pass a flag or a result between domains through a `ref`.	an atomic for a flag; a lock, a queue or a join for anything larger. A value published before an atomic write is visible after an atomic read of the same cell.
10 how many threads	assume a thread is free: ten thousand platform threads hold ten thousand stacks.	a pool, or virtual threads, for many small tasks.	spawn a domain per task.	one domain per core and a queue of tasks, or a library's pool.

Who checks

Rule	Java	OCaml 5
1 every access, or none	nobody. Rust ties the data to its lock, so that there is no way to reach it without holding it.	nobody. Rust ties the data to its lock, so that there is no way to reach it without holding it.

2 two locked steps are not one	nobody. Not Rust either: this program compiles in safe Rust and is wrong in the same way. A data race is what a compiler can rule out; an atomicity violation is ours.	nobody. Not Rust either: this program compiles in safe Rust and is wrong in the same way. A data race is what a compiler can rule out; an atomicity violation is ours.
3 the plain cell is not atomic	nobody. The cost is bounded: a wrong number, never a corrupted heap. In C and C++ the same program has no defined meaning.	nobody. The cost is bounded by the memory model: a wrong number, never a corrupted heap. In C and C++ the same program has no defined meaning.
4 wait in a loop	nobody. The bounded buffer of capacity one, two producers and two consumers, overflows under the `if`.	nobody. The bounded buffer of capacity one, two producers and two consumers, overflows under the `if`.
5 wake everyone	nobody. The same buffer, with the loop but `notify`, stops.	nobody. The same buffer, with the loop but `signal`, stops.
6 one lock order	nobody, and no compiler on the table: Rust compiles this deadlock too. A detector can find it after the fact.	nobody, and no compiler on the table: Rust compiles this deadlock too. A detector can find it after the fact.
7 compare what you read	nobody; the type checker accepts both calls.	nobody; the type checker accepts both calls.
8 a run proves nothing	nobody here. Go's race detector reports a race it saw in a run; Rust refuses the data race before any run; for the atomicity violations of rule 2, nobody.	nobody here. Go's race detector reports a race it saw in a run; Rust refuses the data race before any run; for the atomicity violations of rule 2, nobody.
9 a write another thread must see	nobody. The compiler is what makes the broken loop spin forever: read the language's contract before relying on a plain field.	nobody. The model bounds the race in time, which is why the loop ended here: read the contract before relying on it.
10 how many threads	nobody; the machine answers with memory.	the runtime, by failing.

The runs, side by side

From `python3 compare.py` on this machine, Java 26 and OCaml 5.4.

Rule	Java	OCaml 5	Same behavior

1 every access, or none	broken 754915 of 1000000; fixed 1000000 of 1000000	broken 775100 of 1000000; fixed 1000000 of 1000000	yes
2 two locked steps are not one	broken audits that saw a wrong total: 237619 of 250000; fixed audits that saw a wrong total: 0 of 250000	broken audits that saw a wrong total: 248764 of 250000; fixed audits that saw a wrong total: 0 of 250000	yes
3 the plain cell is not atomic	broken 265937 of 1000000; fixed 1000000 of 1000000	broken 559811 of 1000000; fixed 1000000 of 1000000	yes
4 wait in a loop	broken stuck at the deadline; overflows 13937, underflows 13940; fixed finished; overflows 0, underflows 0	broken stuck at the deadline; overflows 11803, underflows 11879; fixed finished; overflows 0, underflows 0	yes
5 wake everyone	broken stuck at the deadline; overflows 0, underflows 0; fixed finished; overflows 0, underflows 0	broken stuck at the deadline; overflows 0, underflows 0; fixed finished; overflows 0, underflows 0	yes
6 one lock order	broken stuck at the deadline: deadlock; fixed finished	broken stuck at the deadline: deadlock; fixed finished	yes
7 compare what you read	broken compareAndSet(1000, 1001) on a reference holding 1000: false; fixed compareAndSet(v, v + 1) with v the value read: true	broken compare_and_set r (Some 1000) (Some 1001) on an atomic holding Some 1000: false; fixed compare_and_set r v (Some 1001) with v the value read: true	yes
8 a run proves nothing	small 1000 increments per thread: wrong in 1 of 100 runs; large 250000 increments per thread, same program: wrong in 5 of 10 runs	small 1000 increments per domain: wrong in 3 of 100 runs; large 250000 increments per domain, same program: wrong in 10 of 10 runs	yes
9 a write another thread must see	broken the loop never saw the write: stuck at the deadline; fixed the loop ended	ref the loop ended: the racy read saw the write; atomic the loop ended	no

10 how many threads	fixed 10000 threads started and joined: true	broken Domain.spawn failed after 127 domains: failed to allocate domain; fixed Domain.recommended_domain_count () = 12: use that many, and a pool for the tasks	no

Where the two languages part

Point	Java	OCaml 5
The lock	the object's monitor, implicit, `synchronized`	a value, `Mutex.t`; the section is a closure under `Mutex.protect`
The condition	one per monitor, `wait` and `notifyAll`	as many as we make, `Condition.wait c m` and `broadcast`
A field read by one thread and written by another	`volatile`, or an atomic class	an `Atomic.t`; a plain mutable field is a race
What a racy spin loop does	may never end: the compiler keeps the field in a register	ends: the model bounds the race in time
Empty	`null`; the type does not say	`None`; the type says
Compare-and-set compares	identity; by value for `AtomicInteger`	physical equality; by value for `int Atomic.t`
The parallel unit	a thread, thousands of them; virtual threads cheaper	a domain, one per core; `Thread` shares a domain
Shipped structures	`ConcurrentLinkedQueue`, `LongAdder`, `ConcurrentHashMap`	none in the standard library; opam libraries such as Saturn
What a race costs	a wrong value, never a corrupted heap	a wrong value, never a corrupted heap

The code, side by side

1. A lock protects data only if every access takes it

Java

```
class Counter {
  int n;
```

```
  synchronized void inc() { n++; }
  void add(int k) { n += k; }      // forgot the lock
  synchronized int get() { return n; }
}
```

OCaml 5

```
let c = ref 0 and m = Mutex.create ()
let inc () = Mutex.protect m (fun () -> incr c)
let add k = c := !c + k          (* forgot the lock *)
let get () = Mutex.protect m (fun () -> !c)
```

---

2. Two locked steps are not one locked operation

Java

```
// broken: each half under its own lock
synchronized (from) { from.balance -= k; }
synchronized (to)   { to.balance   += k; }

// fixed: both locks, for the whole operation
synchronized (a) { synchronized (b) {
  from.balance -= k; to.balance += k;
} }
```

OCaml 5

```
(* broken *)
Mutex.protect m_from (fun () -> decr from);
Mutex.protect m_to   (fun () -> incr dest)

(* fixed *)
Mutex.protect ma (fun () -> Mutex.protect mb (fun () ->
  decr from; incr dest))
```

---

3. The plain cell is not atomic

Java

```
static volatile int n;
n++;                             // three steps, still a race
```

```
AtomicInteger n = new AtomicInteger();
n.incrementAndGet();              // one instruction
```

OCaml 5

```
let r = ref 0 in
incr r                            (* three steps, a race *)

let a = Atomic.make 0 in
Atomic.incr a                     (* one instruction *)
```

---

4. Wait in a loop, never in an `if`

Java

```
synchronized void put() throws InterruptedException {
  while (count == capacity) wait();   // never: if (...) wait();
  count++;
  notifyAll();
}
```

OCaml 5

```
let put b = Mutex.protect b.m (fun () ->
  while b.count = b.capacity do          (* never: if ... then *)
    Condition.wait b.changed b.m
  done;
  b.count <- b.count + 1;
  Condition.broadcast b.changed)
```

---

5. Wake everyone, unless we can argue for one

Java

```
notify();      // wakes one; a producer may wake a producer
notifyAll();   // wakes all; each re-tests its condition
```

OCaml 5

```
Condition.signal b.changed     (* one *)
Condition.broadcast b.changed  (* all *)
```

6. Take several locks in one order, everywhere

Java

```
// thread 1              // thread 2
synchronized (a) {       synchronized (b) {
  synchronized (b) {}      synchronized (a) {}
}                        }
// fixed: both take a, then b
```

OCaml 5

```
(* domain 1 *)                        (* domain 2 *)
Mutex.protect a (fun () ->            Mutex.protect b (fun () ->
  Mutex.protect b (fun () -> ()))       Mutex.protect a (fun () -> ()))
(* fixed: both take a, then b *)
```

7. Compare-and-set compares what we read, not a value we rebuilt

Java

```
AtomicReference<Integer> r = new AtomicReference<>(1000);
r.compareAndSet(1000, 1001);     // false: a fresh Integer
Integer v = r.get();
r.compareAndSet(v, v + 1);       // true
```

OCaml 5

```
let r = Atomic.make (Some 1000)
Atomic.compare_and_set r (Some 1000) (Some 1001)  (* false *)
let v = Atomic.get r in
Atomic.compare_and_set r v (Some 1001)            (* true *)
```

8. A right answer in a run proves nothing

Java

```
int[] cell = new int[1];
// four threads: for (…) cell[0]++;
```

```
// 1000 each: right in most runs; 250000 each: wrong in most
```

OCaml 5

```
let cell = ref 0
(* four domains: for _ = 1 to per do incr cell done *)
(* 1000 each: right in most runs; 250000 each: wrong in most *)
```

---

9. A write another thread must see goes through an atomic

Java

```
static boolean stop;              // broken: may spin forever
static volatile boolean stop;     // fixed
new Thread(() -> { while (!stop) { } }).start();
```

OCaml 5

```
let stop = ref false              (* a race; here it ends *)
let stop = Atomic.make false      (* what to write *)
Domain.spawn (fun () -> while not (Atomic.get stop) do () done)
```

---

10. How many threads

Java

```
for (int i = 0; i < 10_000; i++) new Thread(task).start();   // fine
Thread.startVirtualThread(task);                             // cheaper still
```

OCaml 5

```
List.init 10_000 (fun _ -> Domain.spawn task)   (* fails after ~127 *)
Domain.recommended_domain_count ()               (* 12 on this machine *)
```